

01-04. gRPC With Codegen

Kita akan build gRPC menggunakan protoc code-gen. Kelebihan dari codegen adalah protoc yang kita buat akan memberikan notif error jika saat kita call client-stub tidak sesuai dengan proto yang kita buat.

Step By Step

1. Install Dependency

Simpan di dev-dependency

```
npm install --save-dev ts-proto grpc-tools
```

2. Modify Employee.proto

- Pastikan file **employee.proto** ada di folder **hr-service-api/protos/employee.proto**.
- Tambahkan function berikut di interface.

```
syntax = "proto3";

package hr;

service EmployeeService {
  rpc GetEmployee(GetEmployeeRequest) returns (EmployeeResponse);

  rpc GetEmployeesForPayroll(GetEmployeesForPayrollRequest) returns
(stream EmployeeResponse);

  rpc ListDepartments(ListDepartmentsRequest) returns (stream
DepartmentResponse);

  // Add: CRUD department dari sisi payroll-service (via hr-proxy)
  rpc CreateDepartment(CreateDepartmentRequest) returns
(DepartmentResponse);
  rpc UpdateDepartment(UpdateDepartmentRequest) returns
(DepartmentResponse);
  rpc DeleteDepartment(DeleteDepartmentRequest) returns
(DeleteDepartmentResponse);

  rpc GetDepartmentWithEmployees(GetDepartmentRequest) returns
(DepartmentWithEmployeesResponse);
}

enum EmploymentStatus {
```

```
    EMPLOYMENT_STATUS_UNSPECIFIED = 0;
    ACTIVE = 1;
    ON_LEAVE = 2;
    SUSPENDED = 3;
    RESIGNED = 4;
    TERMINATED = 5;
}

enum EmploymentType {
    EMPLOYMENT_TYPE_UNSPECIFIED = 0;
    PERMANENT = 1;
    CONTRACT = 2;
    INTERN = 3;
    PROBATION = 4;
}

message GetEmployeeRequest {
    int32 employee_id = 1;
}

message GetEmployeesForPayrollRequest {
    int32 department_id = 1;
    string employment_status = 2;
}

message ListDepartmentsRequest {}

// add
message CreateDepartmentRequest {
    string department_name = 1;
}

message UpdateDepartmentRequest {
    int32 department_id = 1;
    optional string department_name = 2;
}

message DeleteDepartmentRequest {
    int32 department_id = 1;
}

message DeleteDepartmentResponse {
    bool success = 1;
}

message EmployeeResponse {
    int32 employee_id = 1;
    string first_name = 2;
    string last_name = 3;
    string email = 4;
    double salary = 5;
}
```

```
int32 job_id = 6;
int32 department_id = 7;
string employment_status = 8;
string employment_type = 9;
string hire_date = 10;
string updated_at = 11;
}

message ListActiveEmployeesByDepartmentRequest {
  int32 department_id = 1;
}

message DepartmentResponse {
  int32 department_id = 1;
  string department_name = 2;
}

// baru
message GetDepartmentRequest {
  int32 department_id = 1;
}

message DepartmentEmployeePreview {
  int32 employee_id = 1;
  string first_name = 2;
  string last_name = 3;
  string email = 4;
  string employment_status = 5;
}

message DepartmentWithEmployeesResponse {
  int32 department_id = 1;
  string department_name = 2;
  repeated DepartmentEmployeePreview employees = 3; // max 5, preview
  saja
  int32 total_employees = 4;
}
```

3. Create file generator

- Jika struktur project masih menggunakan **node_modules** sendiri.

```
// hr-service-api/script/generate-proto.js
const { execSync } = require('child_process');
const path = require('path');

const isWin = process.platform === 'win32';
const pluginPath = path.join(
```

```
'node_modules', '.bin',
isWin ? 'protoc-gen-ts_proto.cmd' : 'protoc-gen-ts_proto',
);

const cmd = [
  'protoc',
  `--plugin=protoc-gen-ts_proto=${pluginPath}`,
  '--ts_proto_out=./src/grpc/generated',
  '--ts_proto_opt=outputServices=grpc-
js,env=node,esModuleInterop=true,useOptionals=messages,keepCase=true,add
GrpcMetadata=true,addNestjsRestParameter=false',
  '--proto_path=./proto',
  './protos/employee.proto',
].join(' ');

execSync(cmd, { stdio: 'inherit' });
```

- Jika struktur project masih menggunakan sharing **node_modules**.

```
// hr-service-api/script/generate-proto.js
const { execSync } = require('child_process');
const path = require('path');

const isWin = process.platform === 'win32';

// 1. Tentukan path ke root monorepo (naik 2 level dari folder script)
const rootDir = path.resolve(__dirname, '../..');
const serviceDir = path.resolve(__dirname, '..');

// 2. Ambil plugin protoc-gen-ts_proto langsung dari root node_modules
const pluginPath = path.join(
  rootDir,
  'node_modules',
  '.bin',
  isWin ? 'protoc-gen-ts_proto.cmd' : 'protoc-gen-ts_proto'
);

// 3. Tentukan path absolute untuk proto dan output
const protoPath = path.join(serviceDir, 'protos');
const protoFile = path.join(protoPath, 'employee.proto');
const outputDir = path.join(serviceDir, 'src', 'grpc', 'generated');

const cmd = [
  'protoc',
  `--plugin=protoc-gen-ts_proto="${pluginPath}"`,
  `--ts_proto_out="${outputDir}"`,
  '--ts_proto_opt=outputServices=grpc-
js,env=node,esModuleInterop=true,useOptionals=messages,keepCase=true,add
GrpcMetadata=true,addNestjsRestParameter=false',
```

```

    `--proto_path="${protoPath}"`,
    `${protoFile}"`,
  ].join(' ');

  execSync(cmd, { stdio: 'inherit', cwd: serviceDir });

```

- Jika menggunakan mac.

```

{
  "scripts": {
    "proto:generate": "protoc --plugin=protoc-gen-ts_proto=./node_modules/.bin/protoc-gen-ts_proto --ts_proto_out=./src/generated --ts_proto_opt=outputServices=grpc-js,env=node,esModuleInterop=true,useOptionals=messages,keepCase=true,addGrpcMetadata=true,addNestjsRestParameter=false --proto_path=./proto ./protos/employee.proto"
  }
}

```

- File hasil generate akan disimpan di folder **hr-service-api/src/grpc/generated**.

4. Tambahkan script running di package.json

- Tambahkan script proto:generate.

```

"scripts": {
  //others script
  "proto:generate": "node scripts/generate-proto.js"
}

```

- Buka terminal lalu running command berikut :

```
npm run proto:generate
```

5. Modify hr-service.client

Modifikasi code di file **hr-service.client**.

```

import { credentials, status as grpcStatus, type ServiceError } from
'@grpc/grpc-js';
import {
  EmployeeServiceClient,
  type DepartmentResponse,

```

```
    type EmployeeResponse
  } from './generated/employee'; // ini replace function dari hr-types.ts

const HR_SERVICE_GRPC_URL = process.env.HR_SERVICE_GRPC_URL ??
  'localhost:50051';

let client: EmployeeServiceClient | undefined;

function getClient(): EmployeeServiceClient {
  if (!client) {
    client = new EmployeeServiceClient(HR_SERVICE_GRPC_URL,
    credentials.createInsecure());
  }
  return client;
}

export function closeHrServiceClient(): void {
  client?.close();
  client = undefined;
}

export function getEmployee(employeeId: number): Promise<EmployeeResponse |
null> {
  return new Promise((resolve, reject) => {
    getClient().getEmployee({ employeeId: employeeId }, (err:
ServiceError | null, res) => {
      if (err) {
        if (err.code === grpcStatus.NOT_FOUND) return resolve(null);
        return reject(err);
      }
      resolve(res ?? null);
    });
  });
}

export function listEmployeesForPayroll(departmentId?: number,
employmentStatus?: string): Promise<EmployeeResponse[]> {
  return new Promise((resolve, reject) => {
    const results: EmployeeResponse[] = [];
    const call = getClient().getEmployeesForPayroll({
      departmentId: departmentId ?? 0,
      employmentStatus: employmentStatus ?? '',
    });

    call.on('data', (res: EmployeeResponse) => results.push(res));
    call.on('end', () => resolve(results));
    call.on('error', (err) => reject(err));
  });
}
```

```
}

export function streamEmployeesForPayroll(
  departmentId: number | undefined,
  onEmployee: (employee: EmployeeResponse) => Promise<void> | void,
): Promise<void> {
  return new Promise((resolve, reject) => {
    const call = getClient().getEmployeesForPayroll({
      departmentId: departmentId ?? 0,
      employmentStatus: '',
    });

    call.on('data', async (res: EmployeeResponse) => {
      call.pause();
      try {
        await onEmployee(res);
        call.resume();
      } catch (err) {
        call.destroy(err as Error);
      }
    });
    call.on('end', () => resolve());
    call.on('error', (err) => reject(err));
  });
}

export function listDepartments(): Promise<DepartmentResponse[]> {
  return new Promise((resolve, reject) => {
    const results: DepartmentResponse[] = [];
    const call = getClient().listDepartments({});

    call.on('data', (res: DepartmentResponse) => results.push(res));
    call.on('end', () => resolve(results));
    call.on('error', (err) => reject(err));
  });
}

export function createDepartment(departmentName: string):
  Promise<DepartmentResponse> {
  return new Promise((resolve, reject) => {
    getClient().createDepartment({ departmentName: departmentName },
  (err: ServiceError | null, res) => {
    if (err) return reject(err);
    resolve(res!);
  });
  });
}

export function updateDepartment(departmentId: number, departmentName:
  string): Promise<DepartmentResponse> {
  return new Promise((resolve, reject) => {
```

```

        getClient().updateDepartment(
            { departmentId: departmentId, departmentName: departmentName },
            (err: ServiceError | null, res) => {
                if (err) return reject(err);
                resolve(res!);
            },
        );
    });
}

export function deleteDepartment(departmentId: number): Promise<boolean> {
    return new Promise((resolve, reject) => {
        getClient().deleteDepartment({ departmentId: departmentId }, (err:
ServiceError | null, res) => {
            if (err) return reject(err);
            resolve(Boolean(res?.success));
        });
    });
}

```

5. Modify server.ts

Modifikasi file server.ts di folder **src/grpc/server.ts**

```

import path from 'node:path';
import { Server, ServerCredentials } from '@grpc/grpc-js';
import * as protoLoader from '@grpc/proto-loader';
import { EmployeeServiceService, type EmployeeServiceServer } from
'./generated/employee';
import * as employeeGrpcHandler from
'../modules/employees/grpc/employees.grpc-handler';
import * as departmentGrpcHandler from
'../modules/departments/grpc/departments.grpc-handler';
import { ReflectionService } from '@grpc/reflection';

export function startGrpcServer(port = 50051): Promise<Server> {
    return new Promise((resolve, reject) => {
        const server = new Server();

        const handlers: EmployeeServiceServer = {
            getEmployee: employeeGrpcHandler.getEmployee,
            getEmployeesForPayroll: employeeGrpcHandler.getEmployeesForPayroll,
            listDepartments: departmentGrpcHandler.listDepartments,
            // baru
            createDepartment: departmentGrpcHandler.createDepartment,
            updateDepartment: departmentGrpcHandler.updateDepartment,
            deleteDepartment: departmentGrpcHandler.deleteDepartment,
            getDepartmentWithEmployees:
departmentGrpcHandler.getDepartmentWithEmployees

```

```
};

// protoPath digunakan untuk reflection, agar otomatis muncul di grpc
Tools
const protoPath = path.join(__dirname, '../..../protos/employee.proto');
const packageDef = protoLoader.loadSync(protoPath, {
  keepCase: true,
  longs: Number,
  enums: String,
  defaults: true,
});
const reflection = new ReflectionService(packageDef);
reflection.addToServer(server);

server.addService(EmployeeServiceService, handlers);

server.bindAsync(`0.0.0.0:${port}`, ServerCredentials.createInsecure(),
(err, boundPort) => {
  if (err) return reject(err);
  console.log(`[grpc] listening on :${boundPort}`);
  resolve(server);
});
});
}
```

7. Run server

```
npm run dev:otel
```